

MECH5200 Numerical Methods for PDEs : *Video 10:* *2D Finite Difference Equations in MATLAB*

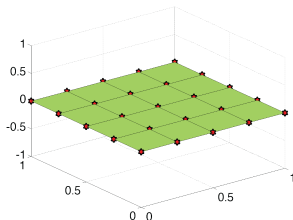
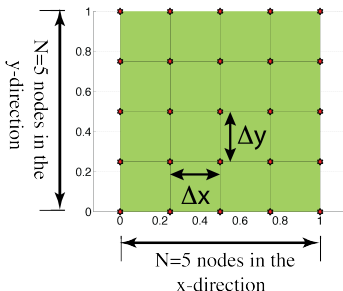
David Willis & Xinfang Jin

February 13, 2026

Step 1: Discretization

- Start by discretizing the membrane:
 - N nodes in the x -direction
 - M nodes in the y -direction:

Total Number of nodes = $N * M = 5 * 5 = 25$



- Total number of nodes = $N * M$.
- $\Delta x = \Delta y = \frac{1}{N-1} \rightarrow N = M$.

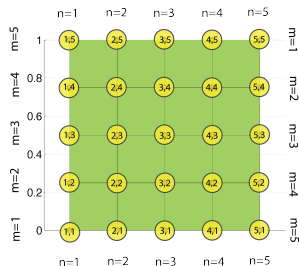
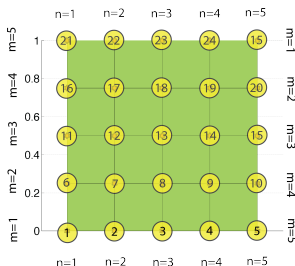
Step 1: MATLAB Discretization – Assuming a Square Domain

- Initial problem setup:
 - We'll assume a square domain ($M=N$, $\Delta x = \Delta y$).
 - We'll use a 'sparse' matrix structure in MATLAB (A-matrix has lot's of zeros)

```
clear all
close all
N = 5; % number of nodes in x
M = N; % number of nodes in y
NNodes = N*M;
DeltaX = 1/(N-1);
% Allocate the matrix A as a sparse matrix in MATLAB
A = spalloc(NNodes,NNodes, NNodes*8);
```

Step 1: Discretization

- To simplify life, we numbered the nodes as:



Here is a MATLAB trick to map the (n,m) -index to node number:

```
counter = 0;
for(m=1:N)
    for(n= 1:N)
        counter = counter + 1;
        NN(n,m) = counter;
    end
end
```

Step 3: Form a Linear System of Equations

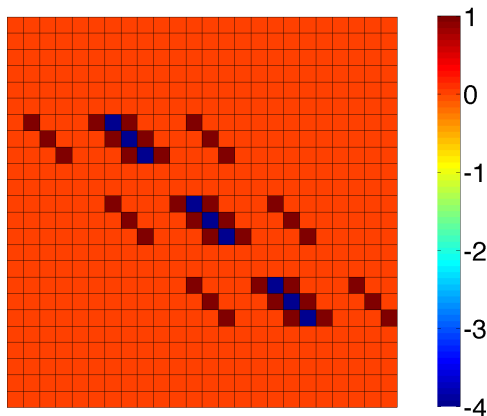
- Each node's finite difference equation has 5-unknowns
- This will become a system of equations (recall multiplication of a matrix and a vector):

$$\text{Node 12: } \frac{1}{(\Delta x)^2} (u_{11} + u_{13} + u_7 + u_{17} - 4u_{12}) \simeq f_{12}$$

$$\frac{1}{(\Delta x)^2} \begin{pmatrix} \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & \vdots & 1 & \dots & 0 & \dots & 1 & -4 & 1 & \dots & 0 & \dots & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix} \begin{pmatrix} u_1 \\ \vdots \\ u_7 \\ \vdots \\ u_{11} \\ u_{12} \\ u_{13} \\ \vdots \\ u_{17} \\ \vdots \\ u_N \end{pmatrix} = \begin{pmatrix} f_1 \\ \vdots \\ \vdots \\ \vdots \\ f_{12} \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ u_N \end{pmatrix}$$

Step 3: Form a Linear System of Equations

- This results in a **linear system of equations**, $Au = f$
- Looking only at the A-matrix for right now:



Step 3: Setting up the A-Matrix

- The A-Matrix setup of internal nodes:
 - Cycle over m, n — internal nodes
 - For each node, insert the appropriate entries (1,1,-4,1,1) in the row of the A-matrix (at correct column locations).
 - Don't forget the $\frac{1}{\Delta x^2}$

```
for(n=2:(N-1))
    for(m=2:(N-1))
        EquationN = NN(n,m);
        % fill in the A-Matrix
        A(EquationN, NN(n-1,m)) = 1;
        A(EquationN, NN(n+1,m)) = 1;
        A(EquationN, NN(n,m-1)) = 1;
        A(EquationN, NN(n,m+1)) = 1;
        A(EquationN, NN(n,m)) = -4;
    end
end
A = A*1/(DeltaX*DeltaX);

% Force A to be sparse
A = sparse(A);
```

Step 3: Form a Linear System of Equations

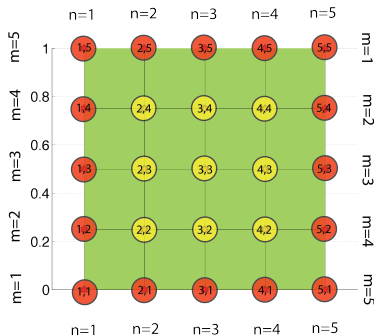
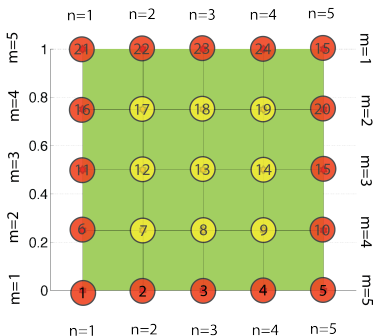
- Don't forget to setup the RHS vector
- for $\nabla^2 u = 1$, we can start with a RHS vector of ones:

```
%Generate the RHS
```

```
b = ones(NNodes,1);
```


Step 4: Incorporate Boundary Conditions

- Introduce boundary conditions.



$$\text{bottom : } u_1 = u_2 = u_3 = u_4 = u_5 = 0$$

$$\text{top : } u_{21} = u_{22} = u_{23} = u_{24} = u_{25} = 0$$

$$\text{left : } u_6 = u_{11} = u_{16} = 0$$

$$\text{right : } u_{10} = u_{15} = u_{20} = 0$$

Step 4: Boundary Conditions

- For the boundary conditions, we will use the node number mapping to go between the node n,m -index and the node-number.
- Recall, we need to update the A-matrix AND the RHS to represent:

$$u_{n,m} = 0$$

- Bottom Boundary node n,m -indexing:
 - $n = 1, 2, 3, 4, 5$
 - $m = 1$

```
% Bottom: Domain Boundary
```

```
for(n=1:N)
    NodeN = NN(n,1);
    A(NodeN, :) = A(NodeN,:)*0;
    A(NodeN, NodeN) = 1;
    b(NodeN) = 0;
end
```

Step 4: Boundary Conditions

- Top Boundary:
 - $n = 1, 2, 3, 4, 5$
 - $m = M$

% Top: Domain Boundary

```
for(n=1:N)
    NodeN = NN(n,M);
    A(NodeN, :) = A(NodeN,:)*0;
    A(NodeN, NodeN) = 1;
    b(NodeN) = 0;
end
```

Step 4: Boundary Conditions

- Left Boundary:

- $n = 1$
- $m = 1, 2, 3, 4, 5$

```
% Left : Domain Boundary
for(m=1:M)
    NodeN = NN(1,m);
    A(NodeN, :) = A(NodeN,:)*0;
    A(NodeN, NodeN) = 1;
    b(NodeN) = 0;
end
```

Step 4: Boundary Conditions

- Right Boundary:
 - $n = N$
 - $m = 1, 2, 3, 4, 5$

% Right Side: Domain Boundary

```
for(m=1:M)
    NodeN = NN(N,m);
    A(NodeN, :) = A(NodeN,:)*0;
    A(NodeN, NodeN) = 1;
    b(NodeN) = 0;
end
```

Solve using MATLAB

- Solve the linear system of equations using MATLAB.

$$\textit{Solution} = A \backslash f \quad (1)$$

```
% Solve the matrix system
x = A\b;
ANSWER = reshape(x, N, M)';
xval = ((1:N)-1)*DeltaX;
yval = ((1:M)-1)*DeltaX;
for(n=1:M)
    XMat(n,:) = xval;
end
for(m=1:M)
    YMat(:,m) = yval';
end
```

Solve using MATLAB

- Plotting: We can use the “surf” command in MATLAB to plot the result (after we reshape):

figure

```
a = axes;  
set(a,'fontsize',20,'fontname','times')  
a = surf(XMat,YMat,ANSWER)  
shading interp  
set(a,'EdgeColor','k')  
caxis([-0.08, 0])  
a = colorbar  
axis equal  
set(a,'fontsize',20,'fontname','times')
```

Solution

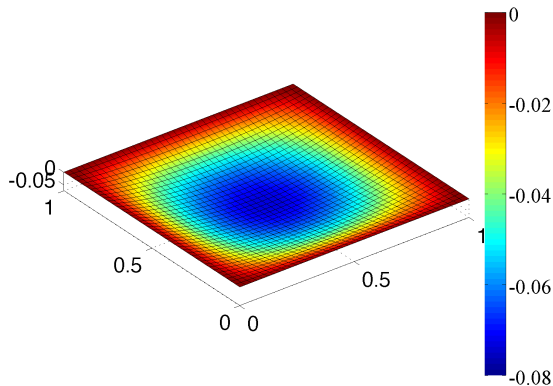


Figure: Solution for $N = M = 41$, $NumNodes = 1681$, Max Deflection = -0.07364

What have we learned?

- The general steps for a finite difference problem are:
 - **Step 1:** Discretize the domain (in 1D: $N - 1$ intervals, N points).
 - **Step 2:** Write finite difference equations that approximate governing equation at each internal node of the problem
 - **Step 3:** Form a linear system ($Ax = b$)
 - **Step 4:** Enforce the boundary conditions where relevant
 - **Step 5:** Solve the system of equations